HANDIAN ARAM
LAM HUGO

SORBONNE UNIVERSITÉ

Rapport projet IA et Sciences des données

Dans le cadre de l'UE Science des Données par l'Exemple, nous avions à réaliser un projet en python proposer par les enseignants. C'est pourquoi, nous, Aram Handian et Lam Hugo avons décidé de s'associer afin de réaliser ensemble le projet. Le but de ce dernier est de réaliser une étude complète d'une base de données. Dans un premier temps, nous allons vous énoncer de quel type de donnée il s'agit, mais également de dévoiler quelques exemples de données dans un graphique. Puis, dans un second temps, nous allons vous exposer les résultats obtenus issus de la base de données à l'aide des outils de réductions de dimensions et de visualisations vus en cours, mais également en TP. Ensuite, nous allons vous présenter les résultats obtenus issus de la base de données grâce à l'outil de classification supervisée. Par la suite, nous allons vous exposer les résultats qu'on a acquis en effectuant un clustering non-supervisée sur la base de données. Enfin, cela nous amènera à une conclusion où l'on apportera ce qu'il nous paraissait le mieux marcher selon chaque tâche.

I. Introduction

Dans cette première partie, nous allons vous énoncer de quel type de donnée il s'agit, mais également de dévoiler quelques exemples de données dans un graphique.

Tout d'abord, dans la première cellule, nous avons importé tous les packages nécessaires au fonctionnement du code. Dans la seconde cellule, à l'aide du code

```
# Cette cellule sert à charger les données
# Si vous avez téléchargé les données, mettez "pickle_mnist=True"
# Sinon, mettez "pickle_mnist=False" et le téléchargement se fera automatiquement, mais cela peut prendre un
# peu de temps

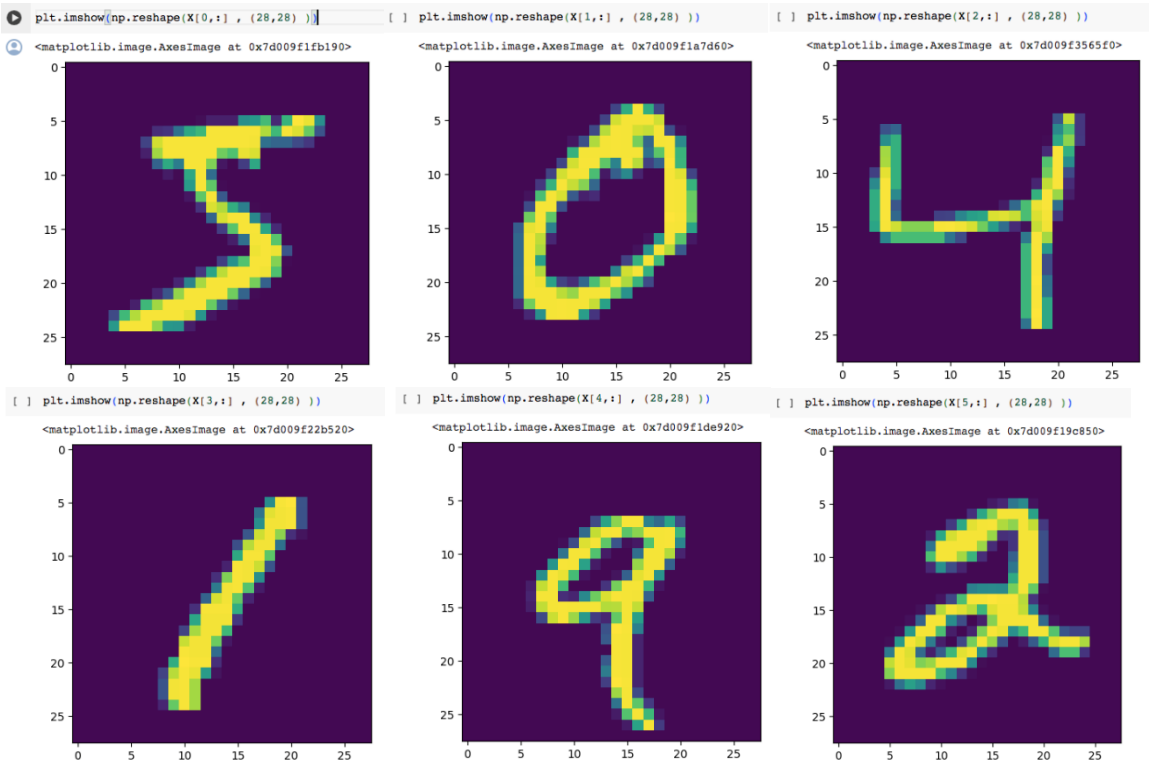
pickle_mnist = False
if(pickle_mnist == True):
    with open('mnist_data.pickle', 'rb') as f:
        digits_database = pickle.load(f)
else: # plus lent
    digits_database = fetch_openml('mnist_784')
```

suivant, nous avons pu importer les données. Ensuite, on réduit la dimension des données. On associe X au data des données et Y au Target. X représente des images des digits (écrit en main) de taille 28*28=784, pour visualiser ces images on utilise

np,reshape qui transforme les vecteurs à des matrices pour qu'on puisse les visualiser :

```
Y[0:6] #pour vérifier si les images affichent les mêmes numéros
```

```
array([5, 0, 4, 1, 9, 2])
```

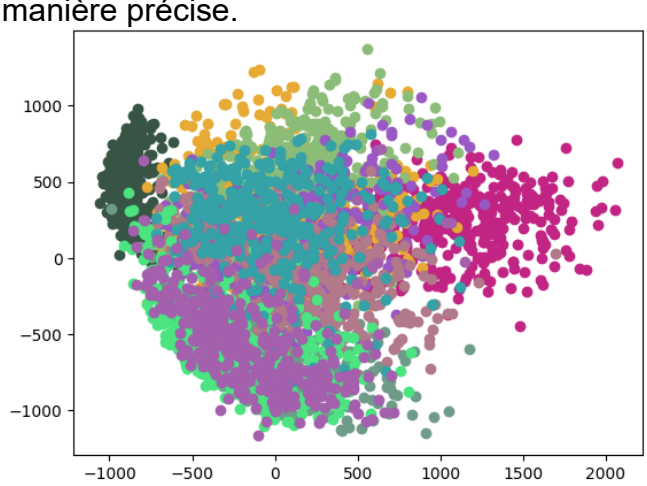


On peut voir que les 6 premiers numéros de Y correspondent aux 6 premières images affichées par X.
Chaque ligne dans X est un digit. Donc il y a 4000 digits. On peut voir que les images représentent les numéros de 0 à 9. Cela correspond à une classification.

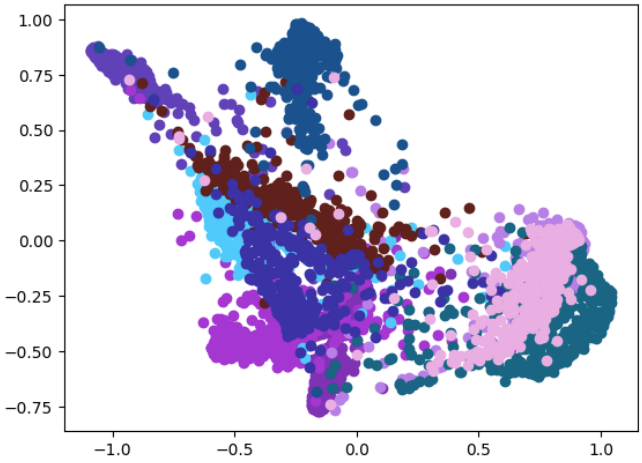
II. Réduction de dimension et Visualisation

Dans cette seconde partie, nous allons vous exposer les résultats obtenus issus de la base de données avec l'aide des outils de réductions de dimensions et de visualisations. La visualisation des données constitue une étape essentielle au début du processus de prétraitement dans le domaine de la science des données. Son rôle est de fournir une représentation graphique des données, permettant ainsi d'appréhender leur structure et leur distribution, même si leur nature intrinsèque ne relève pas nécessairement de la géométrie. En d'autres termes, elle offre une vue d'ensemble visuelle qui facilite la compréhension des caractéristiques fondamentales

des données, contribuant ainsi à une exploration initiale plus approfondie avant d'entreprendre des analyses plus avancées.
Premièrement, nous avons utilisé la méthode « ACP » de réduction de dimension. Et à l'aide de la fonction « def scatter_2d_classes(X,Y): », nous avons affiché le nuage de point. On observe un nuage de point assez désordonné, car la majorité des classes s'entremêlent. C'est pourquoi, nous n'arrivons pas à bien apercevoir les classes de manière précise.



Ensuite, on a fait les mêmes étapes avec l'algorithme t-SNE. Après avoir affiché les points avec la même méthode, on arrive à mieux distinguer les classes que la méthode précédente puisqu'il s'agit d'un algorithme non-linéaire.

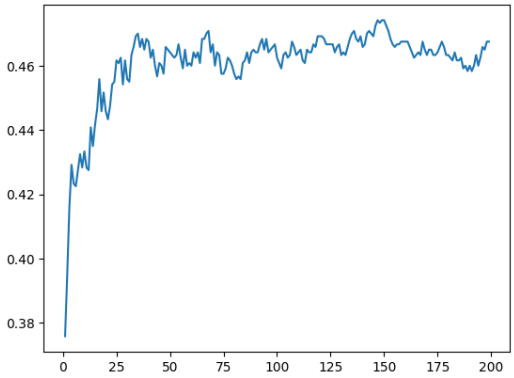


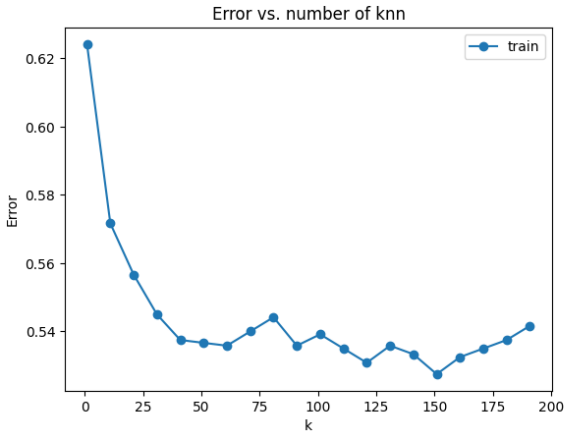
Donc visuellement, on voit que la méthode t-SNE marche mieux**.**

III. Classification supervisée, k-plus-proches-voisins

On sépare les données en deux : entrainement et test. Les données d'entraînement servent à établir le modèle ou entraîner l'algorithme. Ensuite, on évalue l'algorithme sur les données de test. Comme ça, on est sûr que le modèle n'a jamais

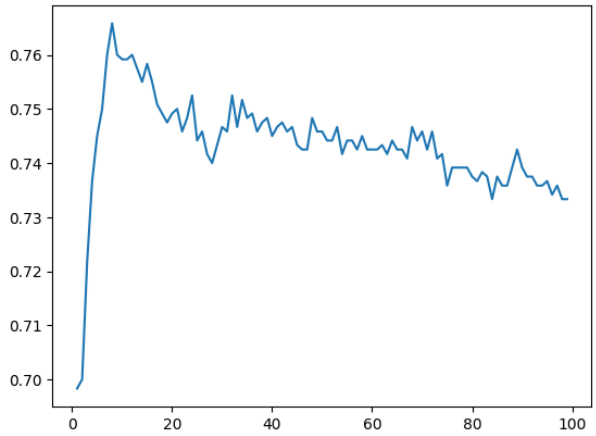
vu les données de test, et un éventuel problème d'overfitting peut se révéler. Pour l'entrainement on prend 70% des données et pour le test on en prend 30%. On a créé 3 fonctions différentes pour chacun des cas, qui sont sans réduction de dimension, avec ACP et avec t-SNE. Comme on utilise la fonction « random.shuffle(inds)» dans les 3 fonctions, donc les données d'entrainement et de test ne sont pas les mêmes dans les 3 cas.
Pour chaque cas on calcule l'accuracy. L'accuracy peut se définir comme le rapport entre les #bonnes détections et les #détections totales. Donc, cela nous permet d'identifier le nombre de fois que l'on a eu bon et par conséquent, la fiabilité d'une méthode. On observe qu'avec la méthode de l'ACP, nous avons obtenu un accuracy d'environ 47%. Ce qui signifie que la méthode de l'ACP n'est pas très fiable. Ensuite, nous avons affiché un graphique qui représente les différentes valeurs de l'accuracy en fonction du nombre de n_neighbors (allant de 1 à 200). On affiche aussi un graphique représentant l'erreur en fonction de n_neighbors qui est k, pour vérifier l'accuracy, car accuracy = 1 – erreur.

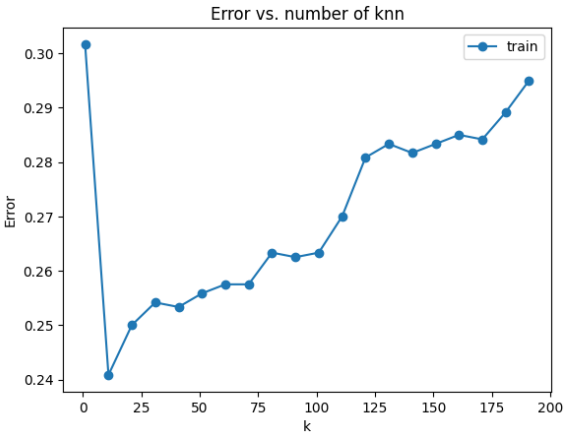




On voit que ces 2 courbes sont inversement proportionnelles. On s'aperçoit que la valeur de l'accuracy ne cesse d'augmenter lorsque le nombre de n_neighbors augmente jusqu'à atteindre 150. Et par conséquent, afin d'obtenir le meilleur accuracy, on choisit n_neighbors = 150 (= k).

On observe qu'avec la méthode t-SNE, nous avons obtenu un accuracy de 76%. Ce qui signifie que cette méthode marche déjà nettement mieux que l'ACP. Puis, pour la méthode t-SNE, nous avons de nouveau affiché un graphique représentant les différentes valeurs de l'accuracy en fonction du nombre de n_neighbors (allant de 1 à 100). On s'aperçoit que les valeurs de l'accuracy ont tendance à diminuer lorsque le nombre de n_neighbors augmente. On observe que l'accuracy atteint son maximum lorsque n_neighbors vaut environ 15. Ainsi, pour la méthode de réduction de dimension appliquée sur ce jeu de données, il est préférable d'utiliser la méthode t-SNE car la méthode t-SNE obtient un meilleur accuracy que la méthode ACP. Ce qui correspond à ce qu'on avait prédit dans la partie II.



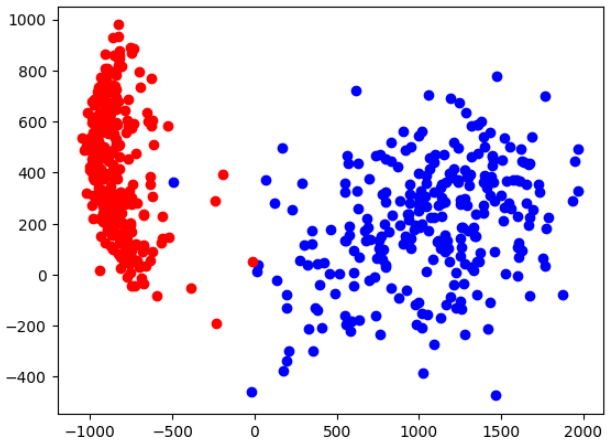


Pour le troisième cas sans réduction de dimension, on écrit cette fonction pour pouvoir visualiser les points des données :

```
d_prime = 2 # pour pouvoir visualiser les données on doit réduire la dimension en 2
pca = PCA(n_components=2)
pca.fit(X_train)
P_train = pca.transform(X_train)
```

```
plt.scatter(P_train[Y_train==0,0],P_train[Y_train==0,1],color='b')
plt.scatter(P_train[Y_train==1,0],P_train[Y_train==1,1],color='r')
```

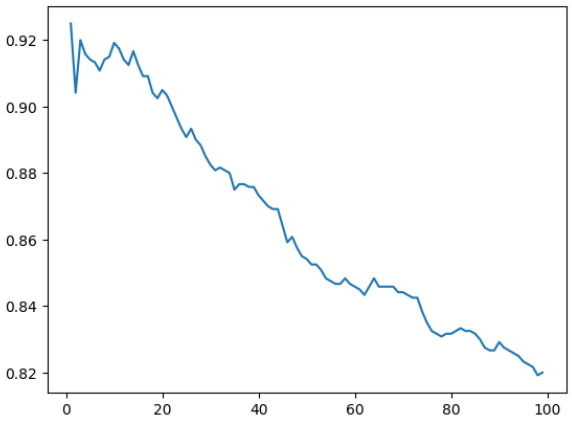
Celle-ci affiche le graphique suivant :

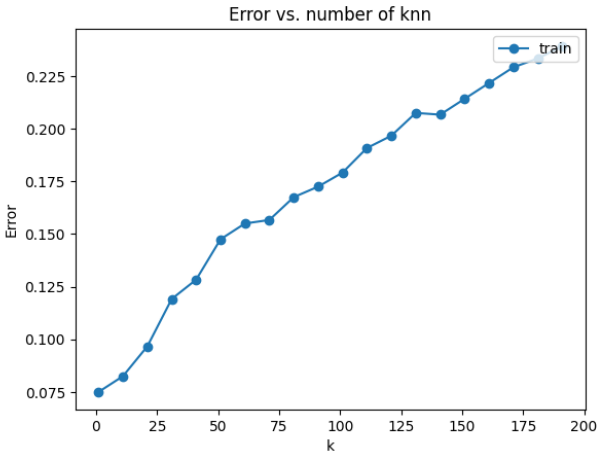


D'après le nuage de point ci-contre, on observe deux classes bien distinctes. Par suite, l'accuracy doit être assez élevé avec la méthode k-plus-proche-voisins (k = 1). Dans ce graphique on observe les indices des digits 1 et 0. On peut aussi faire avec les autres digits (2 à 9) pour visualiser.

Nous avons calculé l'accuracy en initialisant n_neighbors à 1. On obtient un accuracy (ou score) d'environ 93%. On remarque bien que l'accuracy de cette méthode est nettement supérieur aux méthodes précédentes.

Donc, on peut déduire que les prédictions sont presque toutes étaient bonnes. Enfin, nous avons établi un graphique représentant les différentes valeurs de l'accuracy en fonction du nombre de n_neighbors (allant de 1 à 100). On observe que l'accuracy atteint son maximum pour n_neighbors=1. Pour les valeurs n_neighbors suivantes, on aperçoit que la courbe est décroissante. On dessine aussi le graphique représentant l'erreur sur la partie test :





Ainsi, entre les trois méthodes appliquées depuis le début, il est préférable d'utiliser la méthode sans réduction de dimension, car on arrive à bien distinguer les classes sur le nuage de points et on obtient un meilleur accuracy.

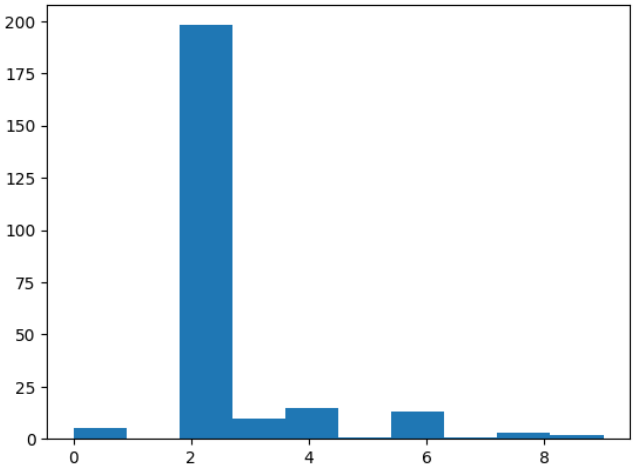
IV. Clustering non-supervisée, k-means

Dans cette quatrième partie, nous allons vous exposer les résultats qu'on a acquis en effectuant un clustering non-supervisée sur la base de données. On effectuera ici un clustering non-supervisée des données grâce à la méthode k-means. Notre but est de déterminer si les clusters trouvés correspondent aux différentes classes de la base de données.

On commence par l'application de l'algorithme de clustering k-means aux données, sans réduction de dimension. On choisit n_clusters 10, parce qu'on veut diviser les digits en 10 clusters de 0 à 9, pour pouvoir les associer après chaque cluster à une classe. Dans la figure ci-dessous on peut voire l'association entre cluster et classe pour les données sans réduction de dimension.

```
[2, 6, 1, 7, 0, 3, 4, 8, 1, 0]
0 correspond à 2
1 correspond à 6
2 correspond à 1
3 correspond à 7
4 correspond à 0
5 correspond à 3
6 correspond à 4
7 correspond à 8
8 correspond à 1
9 correspond à 0
```

On peut explorer chaque cluster en détail en utilisant des histogrammes. Par exemple, concentrons-nous sur le cluster 0, qui est lié à la classe 2 dans la figure ci-dessus.



L'observation révèle que la majorité des éléments sont liés à la classe 2, ce qui suggère également une accuracy assez élevée, étant donné la rareté des associations avec les autres classes.

Ensuite, on évalue les techniques de réduction de dimension. On utilise à nouveau l'algorithme de regroupement k-means sur les données après avoir appliqué l'ACP, puis après avoir utilisé t-SNE. Nous fixons le nombre de clusters à 10 dans les deux cas, comme précédemment. Les associations entre les clusters et les classes sont ensuite affichées pour les deux situations.

Pour l'ACP :

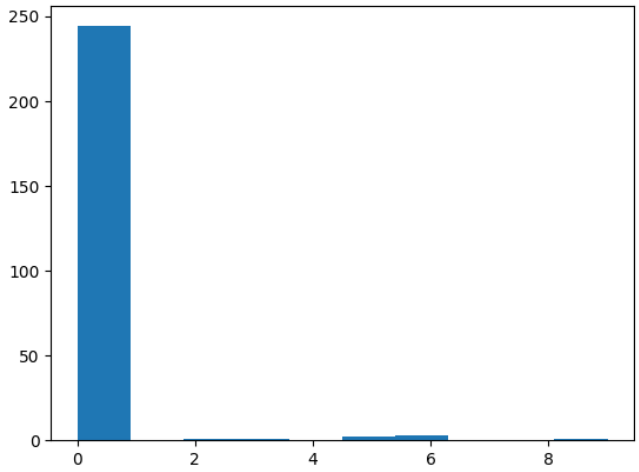
```
[3, 6, 4, 8, 0, 0, 1, 7, 4, 1]
0 correspond à 3
1 correspond à 6
2 correspond à 4
3 correspond à 8
4 correspond à 0
5 correspond à 0
6 correspond à 1
7 correspond à 7
8 correspond à 4
9 correspond à 1
```

Pour le t-SNE :

```
[0, 4, 2, 6, 3, 1, 7, 5, 7, 1]
0 correspond à 0
1 correspond à 4
2 correspond à 2
3 correspond à 6
4 correspond à 3
5 correspond à 1
6 correspond à 7
7 correspond à 5
8 correspond à 7
9 correspond à 1
```

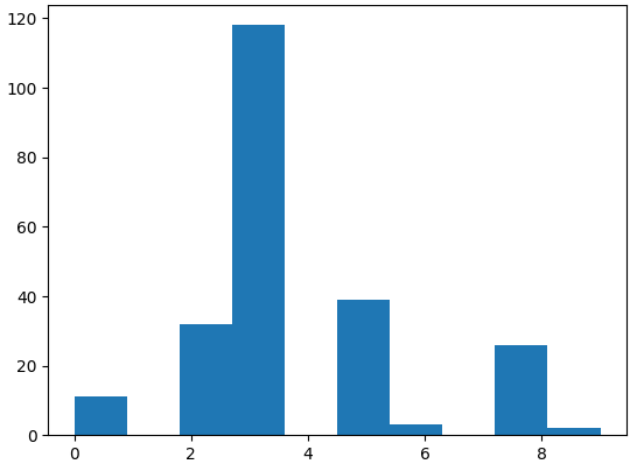
On peut explorer encore une fois chaque cluster en détail en utilisant des histogrammes. Par exemple, concentrons-nous sur le même cluster 0, qui est lié à la classe 3 pour l'ACP et à la classe 0 pour le t-SNE, comme indiqué dans les figures ci-dessus.

Pour le t-SNE :



En observant, on constate que la plupart des éléments sont affiliés à la classe 0. La rareté des liens avec les autres classes suggère également une accuracy élevée. Selon les histogrammes, on peut supposer que la précision (avec t-SNE) est plus élevée que celle sans réduction de dimension.

Pour l'ACP :



En observant, on constate que la plupart des éléments sont affiliés à la classe 0. Cependant, dans ce cas on peut voir plus d'associations avec les autres classes, suggérant une précision potentiellement plus basse que dans les deux autres cas.

En conclusion, en se basant sur les histogrammes, on peut conclure que l'accuracy est la plus élevée dans le cas du t-SNE.

Par la suite, on va calculer l'accuracy de chaque cas pour toutes les données.

On va proposer une méthode automatique pour associer chaque cluster à une classe, et ensuite évaluer les accuracys.

Pour ce faire, on élabore une fonction nommée « def my_accuracy(Y_hat, Y_test) » qui additionne le nombre de prédictions correctes et divise ce total par la taille de l'échantillon.

Avec cette fonction, on trouve l'accuracy avec ACP 43%, sans réduction de dimension 54% et avec t-SNE 61%.

```
my_accuracy(Y_hat,Y_test) # on calcule l'accuracy sans réduction de dimension
```

```
0.5358333333333334
```

```
my_accuracy(Y_hat1,Y_test1) # on calcule l'accuracy avec ACP
```

```
0.43166666666666664
```

```
my_accuracy(Y_hat2,Y_test2) # on calcule l'accuracy avec T_SNE
```

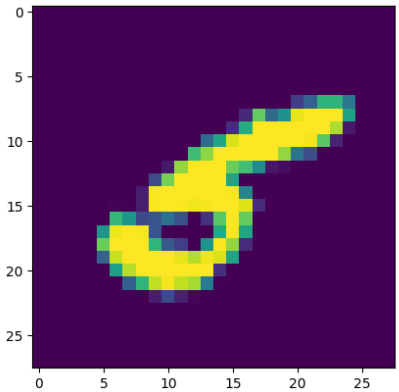
```
0.6091666666666666
```

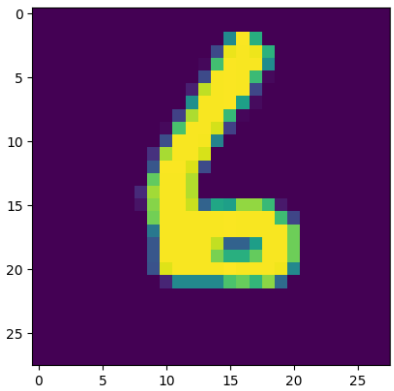
La validité de notre conclusion à partir des histogrammes est confirmée. En ce qui concerne k-means, il est recommandé d'utiliser les données traitées avec t-SNE.

V. Conclusion

En résumé, pour le modèle k-plus-proches-voisins, les données sans réduction de dimension offrent les meilleures performances, car quand on ne réduit pas les dimensions les classes sont bien séparés, mais quand on les réduits en 2 on va voir toutes les classes mélangées et ça serait difficile de regrouper efficacement les classes, tandis que pour le modèle k-means, les données traitées avec t-SNE sont préférables, parce qu'on voit dans la visualisation (partie II) que cette méthode permet de mieux distinguer les classes (parce que c'est une méthode non-linéaire).

Il est notable que les accuracies ne sont pas très élevées, en partie en raison de la similitude visuelle entre certaines images de chiffres, comme celles des chiffres 5 et 6 illustrées ci-dessous.





Ce serait difficile pour les algorithmes à différencier les 2 chiffres dans ce cas.

Ce travail serait plus efficace si les images qu'on utilise étaient en plus grande dimension, et on peut améliorer les résultats en ajustant les hyperparamètres des modèles.

Pour améliorer, on peut aussi essayer toutes les étapes en changeant le pourcentage avec lequel on a divisée les données en entrainement et test.